DocuFlow Project – AI-Powered Documentation Generator

Assignment 6 - Final Document

David Radonic, k11906471

Davyd Pizhuk, k12148477

Table of Contents

Goal of the system

Automatically generate step-by-step documentation (text + screenshots) from user verbally explaining and demonstrating a computer task.

Domain of the system

Productivity Tools and Software Engineering.

Users of the system

Software engineers, IT support staff, technical writers who create tutorials/workflows.

Non-AI goals

- Make creating software documentation fast and easy.
- Provide a simple UI to record, edit, and export guides.
- Support Markdown, HTML, and PDF exports.
- Allow users to manually fix errors before exporting.

AI-related goals

- Transcribe spoken instructions locally for privacy.
- Use a Text LLM to turn raw transcripts into structured JSON steps.
- Use a Vision-Language Model (VLM) to describe screenshots based on context.

Requirements

Functional Requirements

- **Req1**: When the user clicks *Start Recording*, the DocuFlow shall record audio from the system microphone and be ready to capture screenshots on manual command.
- **Req2**: While recording, when the user triggers a manual screenshot, the DocuFlow shall save the image with a timestamp.
- **Req3**: When the user clicks *Stop Recording*, the DocuFlow shall stop audio recording and save the audio file.
- **Req4**: After recording stops, the DocuFlow shall transcribe the audio locally into text using a speech-to-text model.
- **Req5**: After transcription, the DocuFlow shall insert screenshot markers into the transcript using the timestamps.
- **Req6**: Using the transcript, the DocuFlow shall use an LLM to generate structured JSON steps with screenshot placeholders.
- **Req7**: For each screenshot, the DocuFlow shall use a Vision model to write a title and short description.
- **Req8**: After generating descriptions, the DocuFlow shall merge them into the final JSON file.
- **Req9**: When merging is complete, the DocuFlow shall display an editable Markdown preview.
- **Req10**: When the user clicks *Export*, the DocuFlow shall save the file as HTML or PDF.

Non-Functional Requirements

Performance:

- **NfReq1**: After recording, the DocuFlow shall finish transcribing an up to 5-minute audio file within 120 seconds.
- **NfReq2**: During generation, the DocuFlow shall process 10 steps (JSON + Vision) within 90 seconds.
- **NfReq3**: When editing a step, the DocuFlow shall update the preview within 1 second.

Attributes:

- **NfReq4**: The DocuFlow shall achieve at least 95% word accuracy for clear English speech.
- **NfReq5**: The DocuFlow shall score at least 0.70 ROUGE-L compared to human-written reference docs.
- **NfReq6**: The DocuFlow shall allow new users to record and export a task within 5 minutes, without training.

Constraints:

- **NfReq7**: The DocuFlow shall run on Windows 10+.
- **NfReq8**: The DocuFlow shall process all audio locally and never send it to the cloud.
- **NfReq9**: All external API calls (for LLM/VLM) made by the DocuFlow shall use TLS 1.3 encryption.

AI-Related Requirements

AI functional requirements:

- **Req4 - Local Speech-to-Text**
    - Why AI is needed: To avoid manual typing.
    - Threshold: 95% accuracy. (Lower accuracy breaks the LLM's context).
    - Implementation: Use a local open-source model like Whisper.
    - Training: None (use pre-trained).
- **Req6 & Req7 - LLM + Vision Models**
    - Why AI is needed: To structure the text and describe the images.
    - Threshold: 0.70 ROUGE-L on 10+ test workflows.
    - Implementation: Cloud APIs like GPT-4o or Claude 3.5.
    - Training: Zero-shot prompting (no fine-tuning).

AI non-functional requirements:

- **Privacy and security**: Raw audio stays local. Users must consent before text and images are sent to the LLM APIs.

- **Explainability and trust**: The app's preview screen shall display the user's original screenshot side-by-side with the AI-generated step instructions and image descriptions. This allows the user to directly compare the AI's output against their own image to easily spot and fix any mistakes.
- **Safety**: If the AI API crashes, the system will save the raw transcript and images so the user can finish writing manually.

Use case descriptions

Main Use Cases

Use Case 1: Recording and Generating Documentation

Description
Records the user’s spoken explanation and screenshots and generates documentation based on the recorded workflow.

Actors
User, transcription model, vision model, structured JSON creation LLM

Stakeholders
User, readers of the documentation, data protection personnel

Pre-Conditions
Microphone access is available. The user knows how to operate the system and has a clear idea of the workflow they want to document.

Success End Condition
Audio and screenshots are recorded. A transcript and structured documentation are created after the recording, and an editable preview is displayed.

Failure End Condition
The system is unable to create documentation because the recording or processing fails.

Use Case 2: Review and Edit the Generated Documentation

Description
The user reviews the generated documentation and edits content that they want to improve or change.

Actors
User

Stakeholders
User, readers of the documentation, data protection departments

Pre-Conditions
The documentation has already been generated. The editable preview is available, and the user knows how to use the system.

Success End Condition
The updated documentation is saved in the editable preview and is ready for export.

Failure End Condition
The documentation cannot be edited, or the user does not understand how to edit it.

Use Case 3: Export Documentation

Description
Exports the reviewed documentation into a final file format, such as HTML or PDF.

Actors
User

Stakeholders
User, readers of the documentation, data protection departments

Pre-Conditions
The user has saved the documentation and selected the export option in the editable preview.

Success End Condition
The documentation is successfully exported and saved in the selected output format.

Failure End Condition
The documentation cannot be exported.

Use Case 4: Read the Transcript and View the Screenshots

Description
Allows the user to view the saved transcript and screenshots from the recorded workflow in order to understand, verify, manually improve, or write the documentation.

Actors
User

Stakeholders
User, readers of the documentation, supervisor of the user

Pre-Conditions
The recording of the workflow has been completed. The audio has been transcribed, and the transcript and screenshots have been saved.

Success End Condition
The user can access the transcript and screenshots.

Failure End Condition
The transcript or screenshots have not been saved, or the user cannot access them.

Use Case 5: Delete Documentation and Restart

Description
Deletes the current documentation preview together with its related data and starts a new documentation process from the beginning.

Actors
User

Stakeholders
User, DocuFlow software

Pre-Conditions
A documentation preview already exists. The user has access to the current documentation and the restart option.

Success End Condition
The current documentation and related data are deleted, and a new documentation session is started.

Failure End Condition
The current documentation cannot be deleted, or a new session cannot be started.

Supporting Use Cases

Supporting Use Case 1: Transcribe Audio

Description
Converts the recorded audio into a text transcript.

Actors
Transcription model

Stakeholders
User, readers of the documentation, people whose voices are transcribed

Pre-Conditions
A recording session has been completed. The audio file has been saved and is accessible to the system.

Success End Condition
The recorded audio is successfully converted into a text transcript and returned to the system.

Failure End Condition
The audio cannot be transcribed.

Supporting Use Case 2: Record Audio

Description
Records the user's spoken explanation of the workflow through the system microphone.

Actors
User, microphone, transcription model

Stakeholders
User, transcription model

Pre-Conditions
A microphone is connected and accessible. The user has started a recording session.

Success End Condition
The user's audio is successfully recorded and saved by the system.

Failure End Condition
The audio cannot be recorded, or no usable audio file is created.

Supporting Use Case 3: Capture Screenshots

Description
Captures screenshots of the user's workflow on manual command and saves them with timestamps.

Actors
User

Stakeholders
User, readers of the documentation, data protection departments

Pre-Conditions
A recording session is active. The user knows how to capture screenshots and is currently performing the workflow.

Success End Condition
The screenshot is successfully captured and stored with a timestamp.

Failure End Condition
The screenshot cannot be captured or saved correctly.

Supporting Use Case 4: Create Structured Steps

Description
Creates structured documentation steps from the transcript by using an LLM and inserting screenshot placeholders where needed.

Actors
User, structured JSON creation LLM

Stakeholders
User, readers of the documentation, data protection departments

Pre-Conditions
A transcript exists and is accessible. Screenshot markers have already been inserted into the transcript. The system can access the LLM.

Success End Condition
A structured JSON file containing documentation steps is generated and returned to the system.

Failure End Condition
No usable structured steps can be generated from the transcript.

Supporting Use Case 5: Generate Screenshot Descriptions

Description
Generates a title and short description for each captured screenshot by using a vision model and an LLM together with contextual information from the transcript and the generated JSON documentation.

Actors
Vision model, structured JSON creation LLM

Stakeholders
User, readers of the documentation, data protection departments

Pre-Conditions
At least one screenshot has been captured and saved. A transcript exists. The generated JSON documentation steps are available. The system can access the vision model and the LLM.

Success End Condition
A title and short description are generated in JSON format for each screenshot and returned to the system for integration into the documentation.

Failure End Condition
No usable title or description can be generated for one or more screenshots.

Supporting Use Case 6: Display Editable Markdown Preview

Description
Displays the generated documentation as an editable Markdown preview so that the user can review and manually correct the content.

Actors
User

Stakeholders
User, readers of the documentation

Pre-Conditions
A generated documentation draft exists. The Markdown content is available and accessible to the system.

Success End Condition
The generated documentation is successfully displayed as an editable Markdown preview.

Failure End Condition
The Markdown preview cannot be displayed or is not editable.

Supporting Use Case 7: Transform Documentation

Description
Transforms the generated documentation preview into the selected output format, such as HTML or PDF, for export.

Actors
User

Stakeholders
User, readers of the documentation

Pre-Conditions
A generated or edited documentation preview exists. The user has selected an export format.

Success End Condition
The documentation is successfully transformed into the selected output format and is ready to be saved or exported.

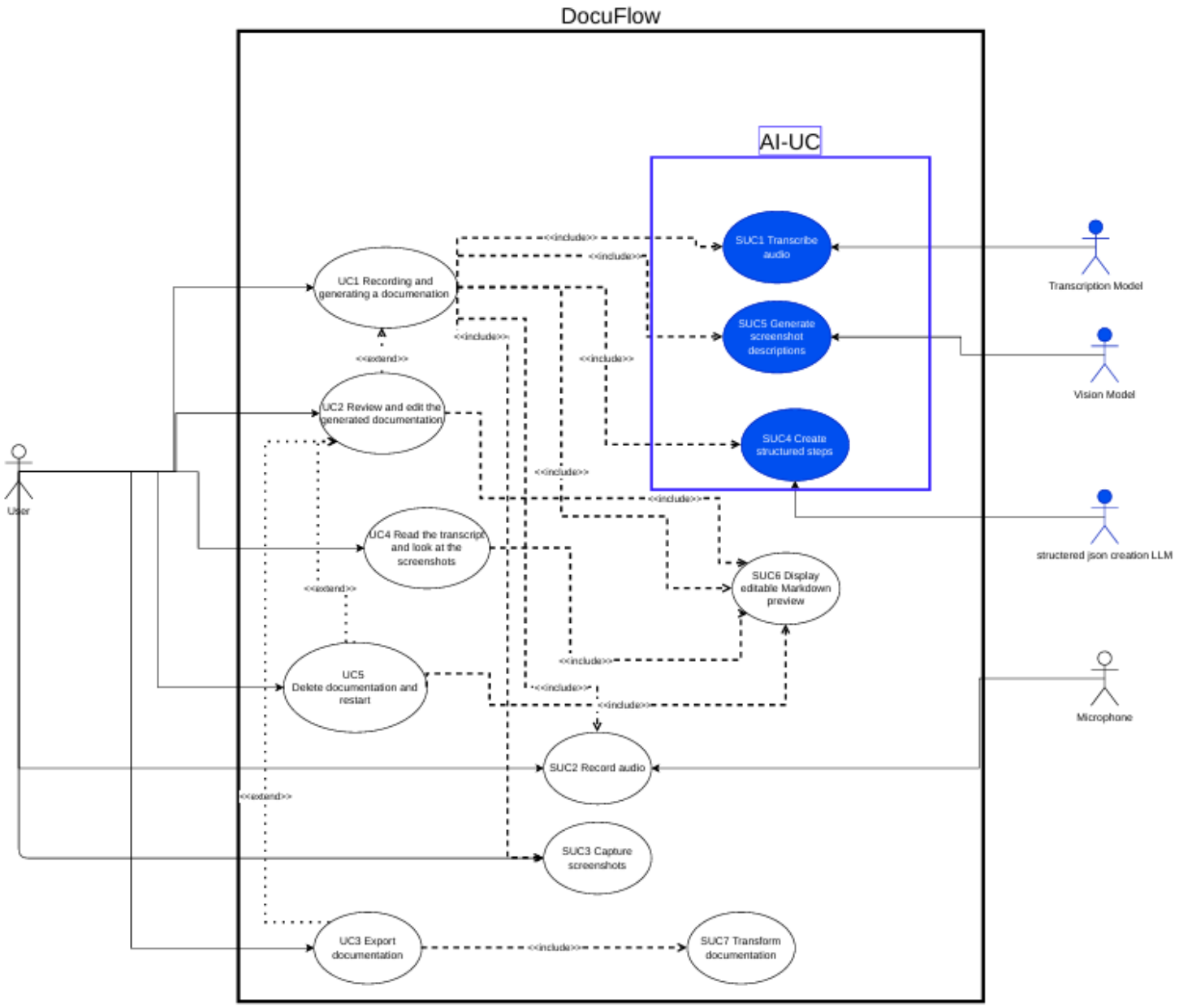
Failure End Condition
The documentation cannot be transformed into the selected format.

Use case diagrams

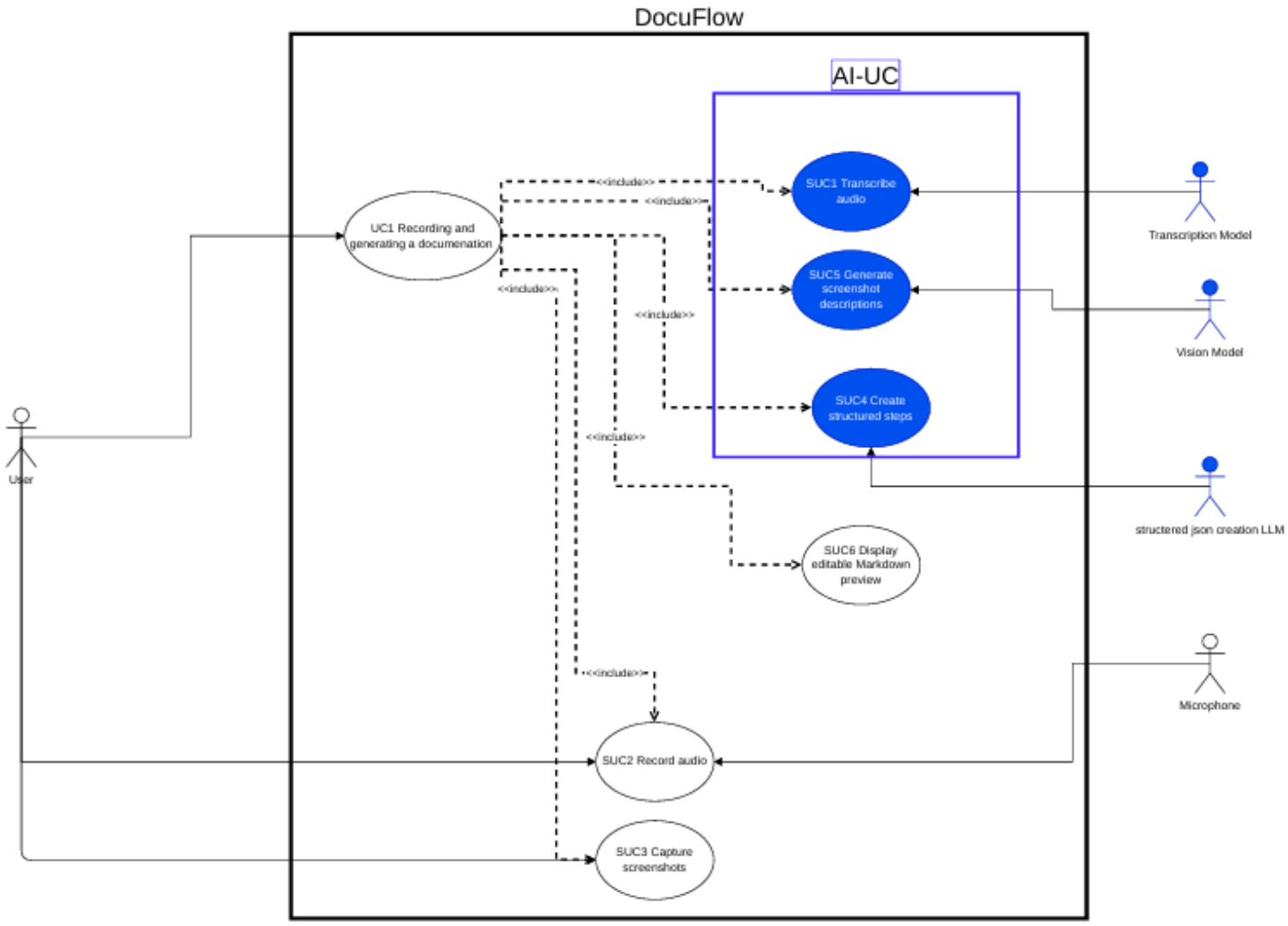
We first present the complete use case diagram to provide an overview of the entire DocuFlow system. To improve readability, we also provide two more detailed diagrams.

Use Case 1 is shown separately because it has the highest number of relationships and includes the central documentation-generation process. The remaining Use Cases 2 to 5 are combined in a second detailed diagram. This structure makes the diagrams easier to navigate and understand.

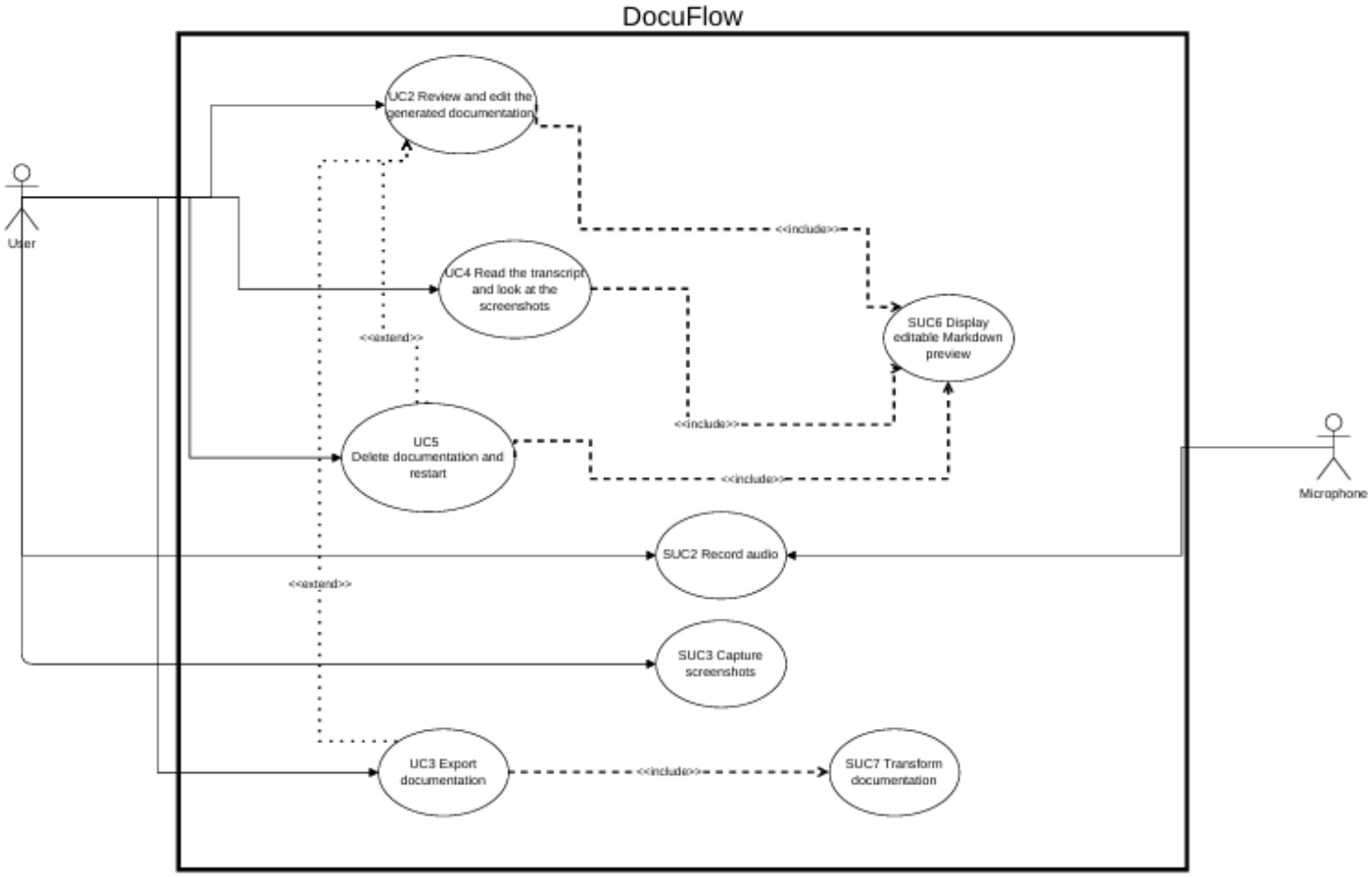
Full Use case diagram



Use case 1 Diagram



Use case 2 - 5 Diagram



Use cases implemented

We implemented all Main & Supporting Use cases in our Prototype.

Main Use Cases

- ✓ **UC1:** Recording and Generating Documentation
- ✓ **UC2:** Review and Edit the Generated Documentation
- ✓ **UC3:** Export Documentation
- ✓ **UC4:** Read the Transcript and View the Screenshots
- ✓ **UC5:** Delete Documentation and Restart

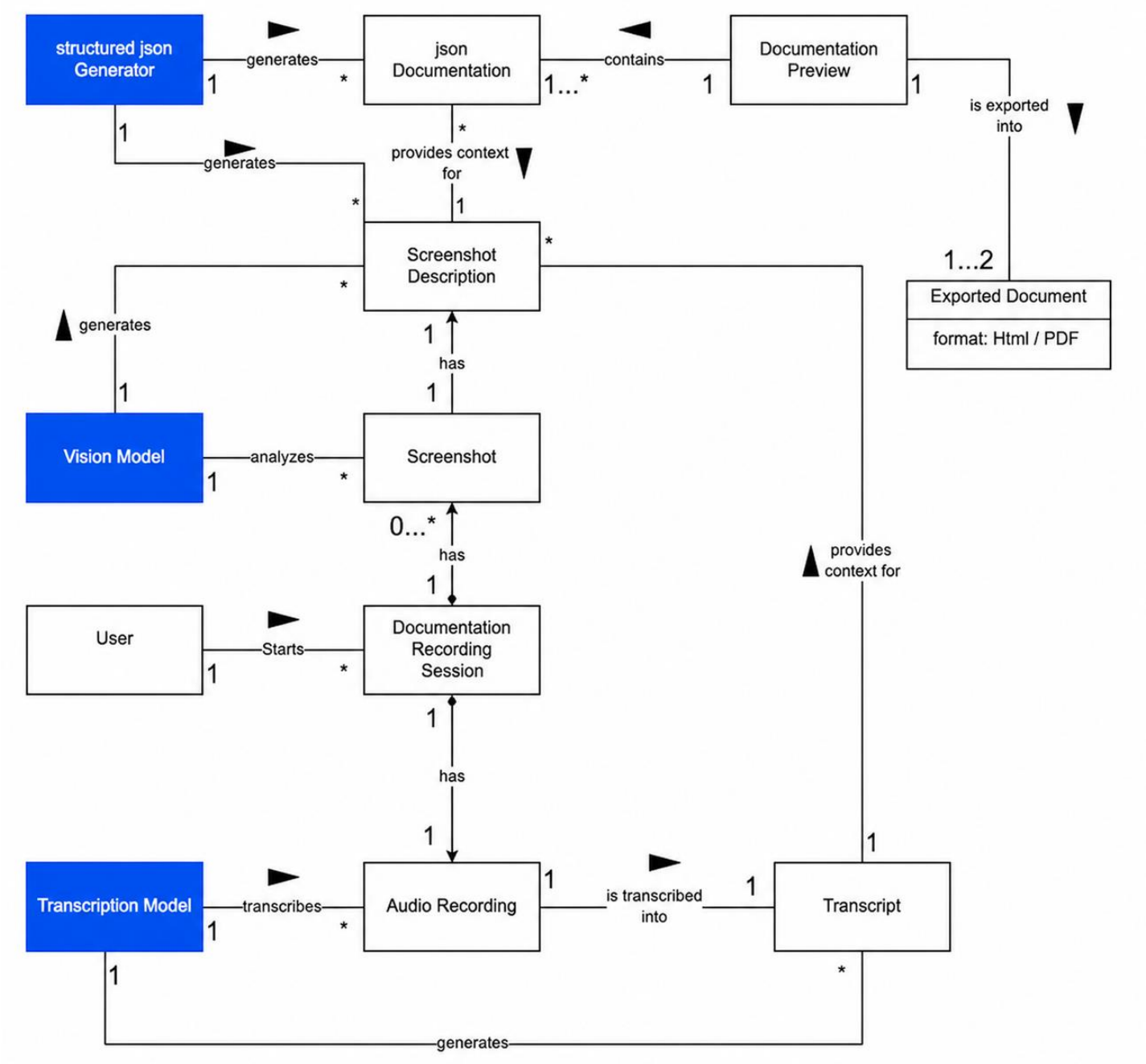
Supporting Use Cases

- ✓ **SUC1:** Transcribe Audio
- ✓ **SUC2:** Record Audio
- ✓ **SUC3:** Capture Screenshots
- ✓ **SUC4:** Create Structured Steps
- ✓ **SUC5:** Generate Screenshot Descriptions
- ✓ **SUC6:** Display Editable Markdown Preview
- ✓ **SUC7:** Transform Documentation

Traceability matrix

Use cases	UC1	UC2	UC3	UC4	UC5	SUC1	SUC2	SUC3	SUC4	SUC5	SUC6	SUC7
Requirements												
Req1	X				X	X	X	X				
Req2	X			X	X			X				
Req3	X				X	X	X	X				
Req4	X			X	X	X						
Req5	X				X							
Req6	X				X				X			
Req7	X				X					X		
Req8	X				X							
Req9	X	X	X		X						X	X
Req10			X									X
NfReq1	X			X	X	X						
NfReq2	X				X				X	X		
NfReq3		X									X	
NfReq4	X			X	X	X						
NfReq5	X			X	X	X				X		
NfReq6	X				X		X		X	X	X	X
NfReq7	X	X	X		X	X	X	X	X	X	X	X
NfReq8	X	X	X		X	X	X					
NfReq9	X				X	X			X	X		

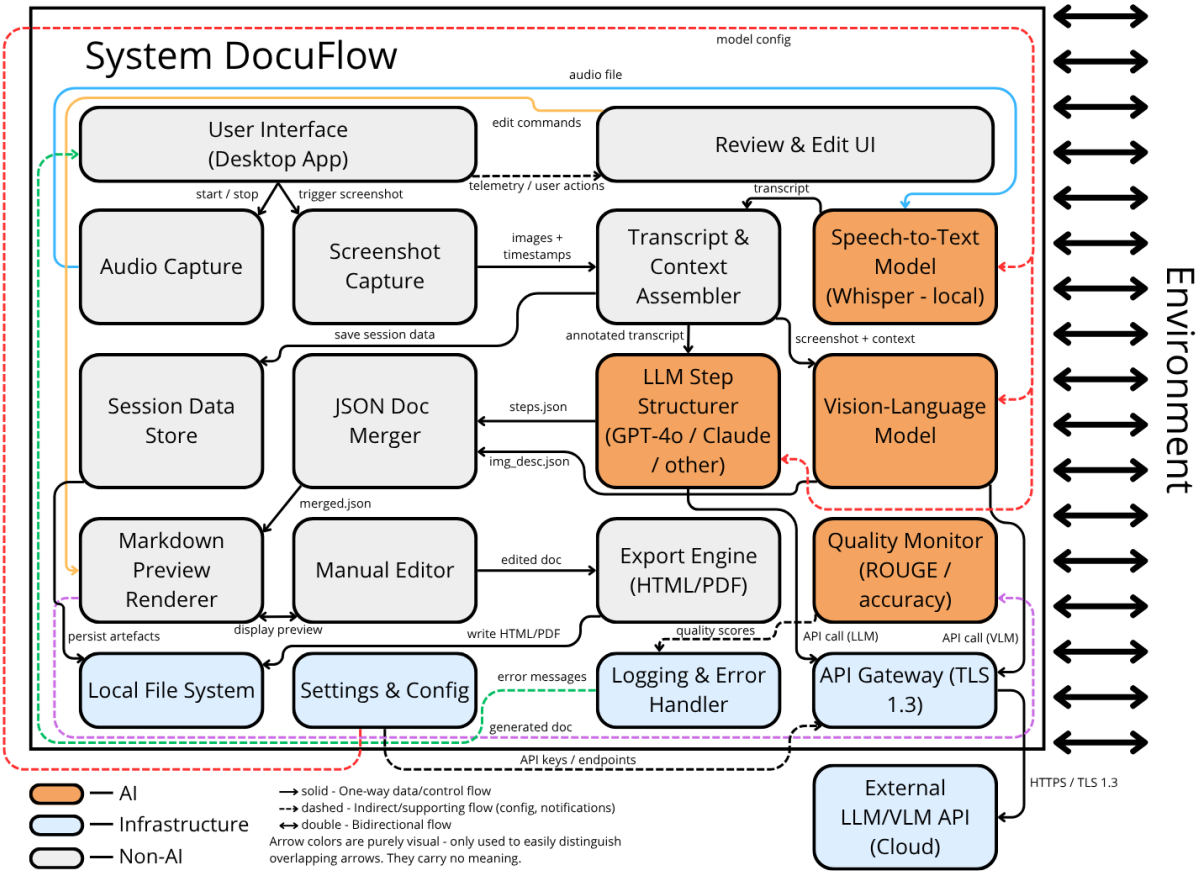
Domain model



Architecture diagram

DocuFlow is a desktop application that records a user's spoken workflow explanation and screenshots, then uses a combination of local and cloud AI models to automatically generate step-by-step documentation.

The diagram below following SysML notation clearly separates **Non-AI components** (gray/blue), **AI/ML components** (orange), and the **System Boundary**



Component Descriptions and Requirements Mapping

Non-AI Components

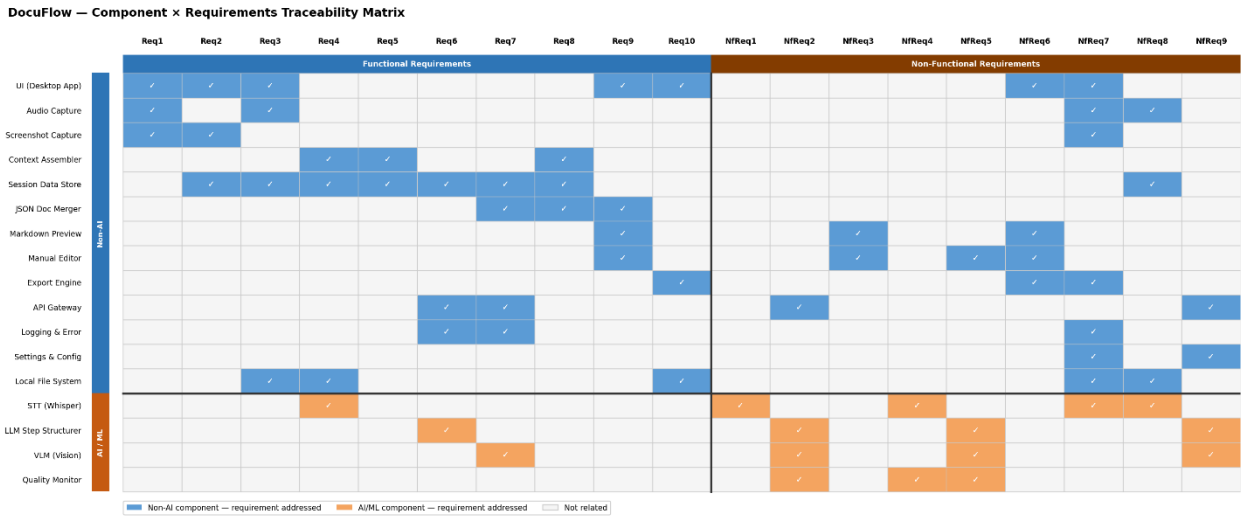
1. **User Interface (Desktop App)**
   The main desktop window exposing Start/Stop Recording, screenshot trigger, preview panel, and export controls. All user-facing feedback (errors, progress) surfaces here.
   Related requirements: Req1, Req2, Req3, Req9, Req10, NfReq6, NfReq7
2. **Audio Capture**
   Records the user's spoken explanation via the system microphone and saves the raw audio file locally - it never leaves the device before transcription.
   Related requirements: Req1, Req3, NfReq7, NfReq8
3. **Screenshot Capture**
   Captures a screen image on manual user trigger, stamps it with a timestamp and saves it locally. The timestamp is later used to place the screenshot at the correct position in the transcript.
   Related requirements: Req1, Req2, NfReq7
4. **Transcript & Context Assembler**
   Merges the finished transcript with screenshot timestamps to produce a single annotated transcript used as input for both the LLM and VLM. All session artefacts also persisted here for fallback access.
   Related requirements: Req4, Req5, Req8
5. **Session Data Store**
   Local folder holding all session artefacts - audio, screenshots, transcript, and intermediate JSON

files. Serves as the manual fallback resource if any AI component fails.
Related requirements: Req2, Req3, Req4, Req5, Req6, Req7, Req8, NfReq8

6. **JSON Doc Merger**
   Combines the structured JSON steps from the LLM and the screenshot descriptions from the VLM into a single unified documentation file. Purely deterministic - no AI involved.
   Related requirements: Req7, Req8, Req9
7. **Review & Edit**
   The screen where the user reads the generated documentation, compares it against the original screenshots, and decides what to change. It connects the Markdown Preview Renderer and the Manual Editor and acts as the single place where the user reviews and approves the output before export.
   Related requirements: Req9, NfReq3, NfReq5, NfReq6
8. **Markdown Preview Renderer**
   Converts the merged JSON into a human-readable Markdown view displayed side-by-side with screenshots, re-rendering within 1 second on any edit. The side-by-side layout is the key explainability mechanism.
   Related requirements: Req9, NfReq3, NfReq6
9. **Manual Editor**
   Text-editing layer embedded in the preview allowing the user to correct, add or remove any step or image description before export. Primary mechanism for fixing transcription or AI errors.
   Related requirements: Req9, NfReq3, NfReq5, NfReq6
10. **Export Engine (HTML / PDF)**
    Transforms the final edited documentation into HTML or PDF based on user selection. One format per invocation; if export fails, a graceful error is shown and the preview stays intact.
    Related requirements: Req10, NfReq6, NfReq7
11. **API Gateway (TLS 1.3)**
    Handles all outgoing calls to cloud LLM and VLM APIs - every request is encrypted with TLS 1.3. Also manages retries and enforces a 60-second timeout, triggering the manual fallback if the cloud is unreachable.
    Related requirements: Req6, Req7, NfReq2, NfReq9
12. **Logging & Error Handler**
    Records all system events and errors to a local log file and surfaces user-visible error messages. Triggers the fallback path (preserving transcript and screenshots) when AI generation fails.
    Related requirements: Req6, Req7, NfReq7
13. **Settings & Config**
    Stores user preferences, model selection, API endpoints, and export defaults. API keys are never stored in plaintext; settings can be changed between sessions without restarting.
    Related requirements: NfReq7, NfReq9
14. **Local File System**
    Underlying OS file system used by the Session Data Store and Export Engine to persist all artefacts and final exports. Provides the durability guarantee enabling manual fallback.
    Related requirements: Req3, Req4, Req10, NfReq7, NfReq8

AI / ML Components

15. **Speech-to-Text Model (Whisper - local)**
    Locally hosted Whisper instance that converts the saved audio file into a plain-text transcript with word-level timestamps, keeping raw audio fully on-device. Must transcribe 5 min in ≤120 s at >=95% word accuracy.
    Related requirements: Req4, NfReq1, NfReq4, NfReq7, NfReq8
16. **LLM Step Structurer (GPT-4o / Claude / other)**
    Cloud LLM called via the API Gateway that converts the annotated transcript into a structured JSON array of documentation steps with screenshot placeholders. Zero-shot prompting, no fine-tuning; must complete 10 steps in <=90 s and score >=0.70 ROUGE-L.
    Related requirements: Req6, NfReq2, NfReq5, NfReq9
17. **Vision-Language Model**
    Cloud multimodal model that generates a title and short description for each screenshot using the image, transcript context, and already-generated JSON steps.
    Related requirements: Req7, NfReq2, NfReq5, NfReq9
18. **Quality Monitor (ROUGE / accuracy check)**
    Optional post-generation component that scores the documentation using ROUGE-L and word-error-rate, displaying results as a plain-language quality badge. Passive only - never modifies output.
    Related requirements: NfReq2, NfReq4, NfReq5



Design Questions

1. What does "good enough" transcription accuracy mean for DocuFlow - and how will we measure it in practice?
2. Will transcribing the same audio twice always produce the same documentation output? Does it matter?
3. How can users tell when the AI got something wrong - and how easy is it to fix?
4. What happens if the LLM API goes down mid-generation? Is any progress lost?
5. How do we make sure no private audio or screenshots leak to the cloud without the user knowing?
6. The LLM sometimes returns broken JSON - what do we do?
7. How do we handle a very long recording where the transcript is too big for the LLM to process at once?
8. How can we tell if a new version of Whisper or the LLM actually produces better documentation?
9. What if the user's microphone records background noise or another person speaking - how does that affect output quality?
10. How do we know the system is working correctly during a live session, and how quickly can we detect and report a failure?

Answers to Design Questions

Q1 - What does "good enough" mean and how do we measure it?
Good enough means the generated documentation is usable without heavy manual corrections. We measure this with two thresholds already in the requirements: >=95% word accuracy for transcription (NfReq4) and >=0.70 ROUGE-L score against human-written reference docs (NfReq5). The Quality Monitor runs these checks automatically after generation and shows the result as a plain-language badge.

Q2 - Will the same audio always produce the same output?
Whisper running locally is deterministic - same audio always produces the same transcript. Cloud LLM and VLM calls are non-deterministic by design, so two runs on the same input may produce slightly different step wording, and that is fine. As long as the output is accurate and usable, minor phrasing variation between runs is acceptable and even beneficial, since it can produce more natural-sounding instructions. We will use a low but non-zero temperature (e.g. 0.3) to balance consistency with natural language quality.

Q3 - How can users tell when the AI got something wrong, and how easy is it to fix?

Every step / space in text in the preview is labeled **"AI"** if it is unchanged from what the LLM generated, or **"Edited"** once the user has modified it, so it is always clear what has been verified and what has not. Users can also open the raw transcript panel next to the preview to compare what was actually said against what the AI wrote, making it easy to spot mistakes. Fixing is straightforward: click any step / place in text, edit it inline, and the preview updates within 1 second.

Q4 - What happens if the LLM API goes down mid-generation? Is progress lost?
No progress is lost. The API Gateway retries up to 3 times with exponential backoff, then writes a checkpoint to the Session Data Store - including the raw transcript, all screenshots, and any partial JSON already generated. An error message tells the user exactly what was saved and where. They can retry the failed step later or complete the documentation manually using those saved artefacts.

Q5 - How do we ensure no data leaks to the cloud without the user knowing?
Audio is transcribed entirely locally by Whisper (NfReq8), it never touches any external server. During installation, the user agrees once to the data policy, which clearly states that transcript text and screenshots will be sent to a cloud LLM/VLM for processing. All cloud calls go through the API Gateway with TLS 1.3 encryption (NfReq9), and if the user does not agree, cloud generation is simply disabled and all artefacts remain local.

Q6 - The LLM sometimes returns broken JSON - what do we do?
All LLM calls will use response_format: json_object (where supported by the API) to enforce valid JSON at the model level. If the output is still malformed, a post-processing validator detects it and automatically retries the call with a corrective prompt explaining the schema error. After two failed retries, the system saves the raw LLM output as a text file so the user can inspect it, and surfaces a clear error message.

Q7 - What if the transcript is too long for the LLM to process at once?
In practice, even a 30-minute recording produces around 4000–5000 words, which fits comfortably within GPT-4o's 128k token context window, so this is unlikely to be a real problem for DocuFlow's target use case. If it does occur, we simply warn the user that the recording is too long and ask them to split it into shorter sessions before processing.

Q8 - How do we know if a new model version actually produces better documentation?
We maintain a small, fixed set of 10 reference workflows - recordings with known, human-written reference documentation - used as a benchmark. Whenever we switch Whisper variant or LLM model, we run the full pipeline on all 10 benchmarks and compare ROUGE-L scores and step counts against the previous version. A new version is only promoted to default if it scores equal or better on at least 8 of the 10 benchmarks.

Q9 - What if the microphone picks up background noise or another speaker?
Whisper is robust to moderate background noise and typically handles it without major accuracy loss. However, a second speaker talking simultaneously is a known failure mode, the transcript may merge their words into the documentation steps. We address this by showing the raw transcript to the user in the Review & Edit UI so they can spot merged or garbled sections before export.

Q10 - How do we detect failures during a live session and report them quickly?
The Logging & Error Handler records every system event with a timestamp: recording started, screenshot captured, API call sent, response received, export completed. Any failure immediately triggers a user-visible error message with a plain-language description and suggested action. Critical failures (microphone lost, API timeout, export failed) are written to a local log file, so issues can be diagnosed later without needing access to the user's private data.